When to Call the Team: Risk-Controlled Escalation for Multi-Agent LLM Pipelines

Anonymous ACL submission

Abstract

Multi-agent pipelines in which LLM agents plan, execute and review can outperform a single agent but often cost several times as much. They are not uniformly better: the team sometimes misses answers a single agent gets right, mostly through wrong first drafts that its reviewer leaves uncorrected. Invoking the team only when a cheaper configuration is likely to fail can cut cost, but a confidence threshold gives no guarantee on the accuracy lost: in our experiments, one of similar average cost lost up to 16 points. We instead minimise cost subject to a user-set tolerance on accuracy loss. CLIMB answers cheaply either directly or after a single agent, as calibration selects, and escalates the rest to the team, stopping on confidence and agreement. It deploys the cheapest policy that a distribution-free test certifies to meet the tolerance with high probability. Since skipping the team can also keep answers the team would miss, the loss is signed, and a variance-adaptive betting test on it typically certifies more than clipping it or applying a Hoeffding–Bentkus bound. Without a certified cheaper policy, CLIMB defaults to the full pipeline, so an unreliable signal tends to reduce savings rather than accuracy. Replayed on three open models and four benchmarks, CLIMB exceeded its tolerance with estimated probability at most 3.4% (the guarantee allows 10%), while saving 38% of tokens on average at a 5-point tolerance and 65% at 10 points.

1 Introduction

Multi-agent systems built from large language models divide a task among roles: a planner decomposes it, an executor solves it and a reviewer critiques the result over several rounds (Du et al., 2024; Wu et al., 2024; Hong et al., 2024). Such pipelines are costly and not uniformly better. On the three open models we study, a plan–execute–review pipeline spends 5.4k to 175k tokens per task, 1.3 to 6.3 times the cost of a single agent that reasons about the task once (Figure 1a), yet some of the tasks it solves are already solved by a direct answer or a single agent (Figure 1b). The team can also reduce accuracy: on some tasks a direct answer or a single agent is correct and the team is not, mostly because its reviewer fails to correct a wrong first draft, and almost half of these losses against the direct answer recur when the team is rerun (§3.1). The relevant question is thus less whether collaboration helps on average than *when it is worth its cost* for a given input.

Invoking the team only when a cheaper configuration is likely to fail can reduce cost, but the difficulty lies in the rule that decides when. At inference time the system observes only weak proxies for the quality of a cheap answer: the mean token log-probability of a direct answer separates correct from incorrect answers with an AUROC of 0.22 to 0.81, falling below chance where greedy decoding produces confident repetitions (Figure 1c). Such proxies can work on average but offer no guarantee. A fixed confidence threshold with the same average cost as our method loses more than 2 points of accuracy on seven of our twelve settings and up to 16 (Figure 1d), and always stopping at the single agent loses up to 9. Without a bound on the accuracy given up, such savings are difficult to trust, and practitioners may reasonably continue to pay for the full pipeline.

We instead minimise expected cost subject to a user-set bound ε on the accuracy lost relative to always running the pipeline. A task climbs a ladder whose rungs are cheaper configurations of the same model, a direct answer L_0 and then a single agent L_1 at the pipeline's own reasoning setting. It stops at a rung once the evidence suffices and escalates to the full pipeline otherwise. Because skipping the team can also retain correct answers the team would miss, this loss is *signed*. When the gain from escalation decreases with the confidence of the cheap answer, the op-

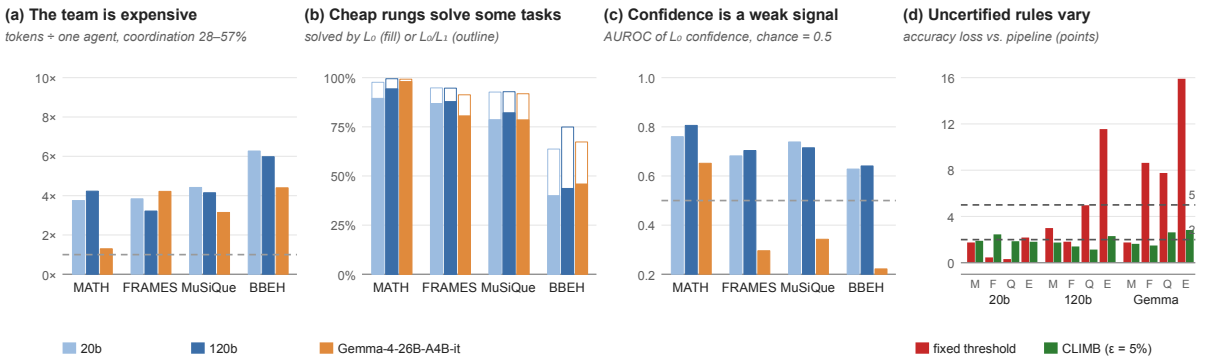


Figure 1: Why escalation needs a guarantee. (a) The team costs up to 6.3 times a single reasoning agent. (b) Some tasks the team solves are already solved by the direct answer L_0 (fill) or by L_0 or a single agent L_1 (outline). (c) Confidence only partly separates right from wrong, and on three settings ranks them backwards. (d) A fixed threshold that spends as much on average as CLIMB exceeds the tolerance on some settings, while CLIMB stays within a tolerance of $\varepsilon = 5\%$ (replay).

timal rule is a confidence threshold, and the safe thresholds are the most conservative ones. CLIMB exploits this ordering without assuming the condition: following Learn-then-Test (Angelopoulos et al., 2025), it tests the thresholds from conservative to aggressive with a betting test on the signed loss (Waudby-Smith and Ramdas, 2024). This yields a distribution-free guarantee: with probability at least $1 - \delta$ for a chosen error level δ, the deployed policy loses at most ε accuracy relative to the team. CLIMB answers cheaply where the guarantee permits and runs the pipeline elsewhere, including on every task when no cheaper policy can be certified, so a weak or misleading signal tends to reduce savings rather than accuracy.

Across three open models and four benchmarks, CLIMB stayed within its tolerance in over 96% of i.i.d. calibration draws, where the guarantee requires 90%, while saving 38% of the pipeline's tokens on average at a 5-point tolerance and 65% at 10 points. Where its signal was misleading, it saved little rather than losing accuracy. The choice of test can matter as much as the signal: on the same records, the Hoeffding–Bentkus bound of Wynn et al. (2026) certifies nothing cheaper than the pipeline, and clipping the loss at zero certifies less on most settings. Our contributions are as follows:

- We measure what a plan–execute–review team adds over a single agent, including the correct answers it loses and where those losses arise, and cast the decision of when to call it as a constrained problem with a signed loss, under which one monotone-gain condition makes a threshold optimal and testing in order lossless (§3, §4.1).
- We introduce CLIMB, a training-free procedure that escalates on confidence and on the agreement of two cheap rungs, and certifies the accuracy lost relative to the team with a variance-adaptive betting test inside fixed-sequence testing (§4.2).
- On three models and four benchmarks we verify that the guarantee holds in practice, show that uncertified rules of similar cost can lose unpredictable amounts, and find that, when most losses are zero, variance adaptivity matters as much as keeping the loss's sign (§5).

2 Related Work

Multi-agent systems, their cost and their failures. LLM agents are organised into debating peers (Du et al., 2024; Liang et al., 2024), conversational frameworks (Wu et al., 2024), role-playing societies (Li et al., 2023) and software teams (Hong et al., 2024; Qian et al., 2024), while self-refinement (Madaan et al., 2023) and reflection (Shinn et al., 2023) are single-agent analogues of the execute-review loop. Efficiency work prunes agents or messages (Zhang et al., 2025; Wang et al., 2025; Chen et al., 2025) or splits a budget across phases (Hamri and Talgam-Cohen, 2026; Liu et al., 2025), typically inside a pipeline that runs on every task. Collaboration can also hurt: debate does not reliably beat simpler prompting once its cost is counted (Smit et al., 2024), self-correction without feedback can turn correct answers wrong (Huang et al., 2024), debaters abandon correct answers to conform (Wynn et al., 2025), and verification failures are a ma-

jor class of multi-agent breakdowns (Cemri et al., 2025). We measure both directions, rescue and harm, and decide per input whether to call the team at all.

Cascades and routing. Cascades defer uncertain inputs to stronger models using learned scores (Chen et al., 2024), self-verification (Aggarwal et al., 2024), trained routers (Ong et al., 2025; Ding et al., 2024) or answer consistency (Yue et al., 2024). DOWN starts a debate only when the first answer's confidence is low (Eo et al., 2025), and Signed Rescue Routing ranks queries by the predicted rescue minus harm of escalating (Wang et al., 2026). None of these bounds the accuracy given up.

Risk-controlled escalation. Conformal prediction and risk control (Vovk et al., 2005; Angelopoulos et al., 2024, 2025) give finite-sample, distribution-free guarantees, applied to generation (Quach et al., 2024; Mohri and Hashimoto, 2024) and to cascades: early exits against a model's full depth with a clipped loss (Schuster et al., 2022), cascades of LLM judges (Jung et al., 2025), delegation to an expert under a non-negative risk (Pona et al., 2026), and absolute accuracy across model tiers (Dou et al., 2026). Signed losses relative to a stronger system have been certified before: Jazbec et al. (2024) bound the gap of early exits with a betting bound (Waudby-Smith and Ramdas, 2024) and show that clipping is conservative, and Wynn et al. (2026) map a signed loss to $[0, 1]$ for Hoeffding–Bentkus tests inside Learn-then-Test. We use such a variance-adaptive betting test on the signed loss inside fixed-sequence testing, add an agreement signal between two cheap rungs, and compare the alternatives directly when most losses are zero, as when escalating to a team (§5.6). Confidence signals (Kadavath et al., 2022; Kuhn et al., 2023; Xiong et al., 2024; Wang et al., 2023) are informative but can be weak, and the guarantee holds for any of them.

3 When Is the Team Worth Its Cost?

3.1 What the team adds

We compare three ways of answering with the same model: a direct answer L_0 at the model's cheapest reasoning setting, a single agent L_1 that reasons once at the pipeline's own setting, and the plan–execute–review team, the *reference* pipeline (setup in §5.1). We first measure what the team buys over the two cheaper configurations.



Figure 2: The team against a single agent L_1 on twelve settings (M, F, Q, E: MATH, FRAMES, MuSiQue, BBEH): share of tasks it rescues (up) and loses (down), with its cost as a multiple of L_1's above.

It often costs several times a single agent. Against the single agent L_1, the pipeline costs 1.3 to 6.3 times as much (Figure 2), 28 to 57% of it in the planner's and the reviewer's own calls.

It rescues some answers and loses others. The team is 0.6 to 9.4 points more accurate than L_1. This net gain is the difference of two larger flows (Figure 2): it rescues 1.2 to 14.9% of tasks that L_1 gets wrong and loses up to 9.3% that L_1 gets right. Rerun on 300 tasks per setting, the team again loses 66 of the 142 tasks it had lost against L_0, so the harms are not only sampling variation. Of the 717 tasks that L_1 answers correctly and the team does not, 84% are already wrong in the team's first draft, and the reviewer, which accepts 23 to 58% of incorrect answers (Figure 9 in Appendix F.3), does not correct them. In the other 16% the reviewer rejected a correct draft and the revision got it wrong.

Choosing per task could recover the losses. Since the team is the more accurate of the two everywhere, these losses, up to 9.3 points, are the most that choosing per task between the two could add. The ladder answers cheaply where the cheap rungs are likely right and escalates the rest, within a certified tolerance.

3.2 Escalation as a constrained decision

A task x climbs a ladder of cheap rungs $k = 0, \ldots, K-1$ of increasing cost, cheaper configurations of the same model that stop short of the full pipeline (here $K = 2$: L_0 and L_1). Rung k returns an answer with correctness $Y_k \in \{0, 1\}$ and a score S_k, a fixed function of the outputs so far, and stopping there costs the cumulative C_k. After the last cheap rung the system *escalates* to the full pipeline, the *reference*, with correctness Y_{ref} at additional cost C_{ref}. A policy π either runs the refer-

ence directly ($\varnothing$) or climbs and stops according to the scores. Its risk is the accuracy it gives up relative to the reference, $R(\pi) = \mathbb{E}[Y_{\text{ref}} - Y_\pi]$, and its cost is $J(\pi) = \mathbb{E}[C_\pi]$, for the correctness Y_π and total cost C_π of its answer on a deployment task, with $R(\varnothing) = 0$. A policy $\hat{\pi}$ fitted on n labelled calibration tasks is (ε, δ)-*risk-controlled* if

$$\Pr(R(\hat{\pi}) > \varepsilon) \leq \delta, \tag{1}$$

over the calibration sample: except with probability δ, it gives up at most ε accuracy relative to the pipeline. The *constrained problem* is

$$\min_{\pi} \; J(\pi) \quad \text{s.t.} \quad R(\pi) \leq \varepsilon, \tag{2}$$

strictly feasible for every $\varepsilon > 0$ since $\varnothing$ has zero risk.

One decision. With a single stopping point on a score S, a policy stops with probability $\pi(s)$, its risk is $R(\pi) = \mathbb{E}[(Y_{\text{ref}} - Y_c)\,\pi(S)]$ for the cheap answer's correctness Y_c, and two quantities govern the problem, the accuracy that escalating buys and the tokens that stopping saves:

$$g(s) = \mathbb{E}[Y_{\text{ref}} - Y_c \mid s], \quad c(s) = \mathbb{E}[C_{\text{ref}} \mid s]. \tag{3}$$

The gain g can be negative where the pipeline misses more answers the cheap rung gets right than it fixes, while the saving c is positive.

4 Method

CLIMB solves the constrained problem in Eq. (2) in two steps. A structural step (§4.1) shows that, under one condition on the gain, the optimal rule is a threshold on the score. A statistical step (§4.2) selects the threshold by testing candidates in order, with a guarantee that needs no such condition. §4.3 instantiates the ladder (Figure 3).

4.1 When a threshold is optimal

We first take the one-decision setting of §3.2. The condition is that escalation buys less the more confident the cheap answer is.

(M) *Monotone gain*: $g(s)$ is non-increasing in s, and so is $g(s)/c(s)$, the accuracy bought per token of escalation.

Proposition 1 (Threshold optimality)**.** *With one decision and under (M), for every $\varepsilon > 0$ the constrained problem in Eq. (2) has an optimal policy that is either $\varnothing$ or stops when $S > t$ and escalates when $S < t$, for some threshold t. If S has a continuous distribution, this policy is deterministic.*

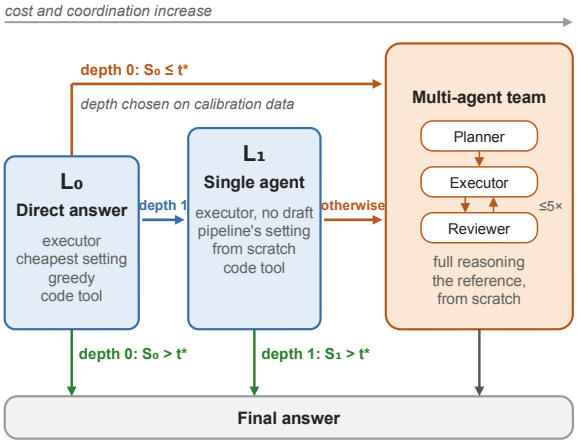


Figure 3: The ladder. Calibration picks one depth: stop at the direct answer L_0 when it is confident, or run both cheap rungs and stop at the single agent L_1, agreeing tasks first, each group by L_1's confidence. Every other task runs the full pipeline from scratch.

The constrained problem is a fractional knapsack whose items are score values, weighted by the risk $g(s)$ of stopping there and valued by the cost $c(s)$ it saves. Under (M), filling it greedily means stopping above a threshold (Appendix A): a task is answered cheaply when the accuracy at stake is worth less than the tokens escalation would spend. Each policy family CLIMB certifies (§4.2) makes exactly one decision, after L_0 or after both cheap rungs, so the proposition applies to each family separately, and we test (M) for S_0 (Appendix E.1).

4.2 Risk-controlled calibration

The optimal threshold depends on the unknown g and c. Rather than estimating them, we select it with a distribution-free procedure. Following Learn-then-Test (Angelopoulos et al., 2025), each candidate threshold becomes a null hypothesis that its risk exceeds the tolerance, and a threshold is deployable only if the calibration data reject that hypothesis.

Candidates. $\Lambda = (\varnothing, t_1, \ldots, t_M)$ with $t_1 > \cdots > t_M$, fixed before any calibration label is seen and ending at $t_M = -\infty$, which never escalates. Lower thresholds stop more tasks, so the sequence runs from conservative to aggressive and from expensive to cheap. The action $\varnothing$ has zero risk, so it is always admissible.

Testing the loss itself. The per-task loss of threshold t, $\ell = Y_{\text{ref}} - Y_{\pi_t} \in \{-1, 0, 1\}$ for π_t, which stops when $S > t$, is signed: stopping can

lose an answer the pipeline gets right, or keep one the pipeline would get wrong. We test its mean directly with a betting test (Waudby-Smith and Ramdas, 2024). Under $\mathcal{H}_m$, that the loss of t_m has mean above ε, the wealth of a gambler who bets that the mean loss is below ε,

$$W_j = \prod_{i \le j} \big(1 + \lambda_i(\varepsilon - \ell_i)\big), \quad 0 \le \lambda_i < \tfrac{1}{1-\varepsilon}, \tag{4}$$

is a nonnegative supermartingale whenever each λ_i depends only on earlier tasks, so by Ville's inequality (Ville, 1939) $p_m = 1/\max_j W_j$ is a valid p-value. We set λ_i from the running variance of the earlier losses, which makes the test adapt to how often stopping changes the answer at all.

Sequential testing. On n calibration tasks, each with every cheap rung's and the reference's outcomes recorded, we test $\mathcal{H}_1, \mathcal{H}_2, \ldots$ in order and stop at the first m with p_m above the level of the test ($\delta/2$ per family below). The admissible set $\mathcal{C}$ contains $\varnothing$ and every rejected threshold, and CLIMB deploys its cheapest member (Algorithm 1 in Appendix C).

Choosing the depth. Whether the second rung pays for itself depends on the model and the task, so CLIMB lets the calibration data choose the depth. It runs the test above on two one-decision families, a threshold on L_0's score ($D = 0$) and, after both cheap rungs, a threshold on S_1, which combines the two rungs' agreement with L_1's confidence ($D = 1$), each at level $\delta/2$ with admissible sets $\mathcal{C}_0$ and $\mathcal{C}_1$, and deploys the cheaper of the two certified policies. By a union bound both certificates hold at once with probability at least $1 - \delta$, so the choice costs nothing in validity.

Proposition 2 (Risk control)**.** *If calibration and deployment tasks, together with the model's outputs on them, are i.i.d., then with probability at least $1 - \delta$ every policy in $\mathcal{C}_0 \cup \mathcal{C}_1$, including the deployed one, has $R \le \varepsilon$. The deployed policy's mean calibration cost is at most the pipeline's.*

Testing in a fixed order needs no correction for the number of candidates, because a true null can be rejected only if the first true null in the sequence is, which happens with probability at most the level. On the event that none is, every certified policy is safe, so deploying the cheapest costs nothing (Appendix A).

Proposition 3 (Efficiency)**.** *Let $r(t) = R(\pi_t)$ and $J(t)$ be the risk and expected cost of the threshold t. With one decision and under (M): (i) the thresholds with $r(t) \le \varepsilon$ form a prefix $t_1, \ldots, t_{m_\varepsilon}$ of the sequence, (ii) J does not increase along the sequence, so t_{m_ε} is the cheapest safe threshold, and (iii) for every $\eta > 0$, the threshold* CLIMB *deploys has expected cost at most $J(t_{m_{\varepsilon-\eta}}) + o_P(1)$ as $n \to \infty$.*

Part (i) is what makes testing in order lose nothing: stopping at the first failure discards no safe threshold. With Proposition 1, (iii) says that CLIMB's cost approaches, up to the grid's resolution, that of the best one-decision policy on the same score as calibration data grow (Appendix A).

What the guarantee covers. Proposition 2 needs neither (M) nor a calibrated score: a weak or even misleading signal tends to cost savings rather than accuracy beyond the tolerance. It bounds the loss relative to the pipeline, not to the best achievable system, and its i.i.d. assumption covers the model's outputs as well as the tasks. We set $\delta = 0.1$ and give the threshold grid in Appendix C.

4.3 Instantiation

The ladder. Two cheap rungs use the same model as the pipeline. L_0 answers directly at the model's cheapest reasoning setting (low effort, or thinking off where the model has no graded setting). L_1 is a single agent at the pipeline's own reasoning setting: it answers the question from scratch, without L_0's answer, as the pipeline's executor would without a plan or a review. Every rung's call ends with a line `ANSWER: <answer>` and may run Python (Appendix B).

The pipeline. An escalated task runs the reference from scratch: a planner writes a step-by-step plan, the executor answers given the plan, and a reviewer judges the answer, ending with a line `VERDICT: YES` or `NO`. A rejection prompts a revision, for at most five rounds. Its context is rebuilt each round from the question, plan and only the latest answer and review. Tasks have no per-task token budget, so the harms we measure are unlikely to be artefacts of truncation.

The score. S_0 is the mean token log-probability of the call that produced L_0's answer, available at no extra cost. After both rungs, the score adds whether they agree: $S_1 = 10 \cdot \mathbf{1}[a_0 = a_1] + \bar{s}_1$, where a_k is rung k's normalised answer and $\bar{s}_1$ the mean log-probability of L_1's final call. Since L_1 answers without seeing L_0's answer, agreement

between them is evidence that costs nothing beyond the two rungs. As $-10 < \bar{s}_1 \leq 0$ on all our records, every task on which the two rungs agree ranks above every task on which they do not, and the threshold sequence first stops agreeing tasks in order of L_1's confidence.

5 Experiments

5.1 Setup

Models and benchmarks. We use gpt-oss-20b and gpt-oss-120b (OpenAI, 2025) and Gemma-4-26B-A4B-it (Gemma Team, 2026), a mixture-of-experts model. L_0 uses each model's cheapest reasoning setting (low effort for gpt-oss, thinking off for Gemma) with greedy decoding, while L_1 and the pipeline use the default setting (medium effort, thinking on) at temperature 1.0. We evaluate on MATH-500 (Hendrycks et al., 2021; Lightman et al., 2024), FRAMES (Krishna et al., 2025), answerable MuSiQue (Trivedi et al., 2022) and BIG-Bench Extra Hard (BBEH, Kazemi et al., 2025): test splits of 400, 660 and 1,934 tasks for the first three, whose other 20% served to build the pipeline, and 874 BBEH problems (38 from each of its 23 subtasks), which played no part in it. FRAMES and MuSiQue are open-book: every call receives the question's documents (median 17k and 2.4k tokens), so every rung pays for them. MATH is scored by symbolic equivalence, BBEH by its official fuzzy match, and FRAMES and MuSiQue by an LLM judge (gpt-oss-120b). Cost is prompt plus completion tokens, including hidden reasoning and tool calls, over all calls a task makes. Model settings, documents, scoring and generation caps are detailed in Appendix B.

Records and replay. For every task we record the reference pipeline and CLIMB's ladder forced through every cheap rung, which gives the outcome, cost and confidence of L_0 and L_1. Replaying these records evaluates any policy on any subset of tasks. An escalating task receives the reference's outcome and pays its cost on top of the rungs already run.

Protocol and baselines. Losses are measured against the reference, and tolerances are absolute: $\varepsilon = 10\%$ allows a loss of 10 accuracy points. We draw 200 random calibration/evaluation splits (50/50, or 100 splits of 20/40/40 for policies that need training data) and report means. The *violation rate* is the share of splits in which the selected policy's loss on the full pool exceeds ε. We compare with uncertified rules (always stopping at L_0 or at L_1, and a single confidence threshold, fixed across settings, that spends as much as CLIMB on average), an oracle that stops at the cheapest correct rung, and richer escalation policies certified the same way (Appendix F.2).

We ask whether the guarantee holds (§5.2), what a tolerance buys (§5.3), what limits the savings (§5.4), how the score and the reference matter (§5.5), and which ingredients make certification efficient (§5.6).

5.2 The guarantee holds

Proposition 2 bounds the probability, over the calibration sample, that the deployed policy loses more than ε. We check this event directly by treating each recorded pool as the task distribution, drawing calibration sets i.i.d. from it and computing the deployed policy's exact risk: over 5,000 draws per setting and tolerance the failure rate is at most 3.4% (upper 95% limit 3.9%) against $\delta = 10\%$ (Appendix D). The finite-pool violation rate of the replay agrees: at most 2% in Table 1 and 3% anywhere in the tolerance sweep.

The guarantee sets these savings apart from those of simple rules (Table 2). Their losses can be modest on average but large on some settings, with nothing at deployment to say which: always stopping at L_0 loses up to 20.6 points, at L_1 up to 9.4, and a threshold on L_0's confidence that spends as much as CLIMB on average up to 15.9. Three of the four settings where it loses over 5 points are those where Gemma's confidence is inverted (Figure 1d). Tuned per setting without certification, the threshold exceeds $\varepsilon = 5\%$ on 31% of splits on average (Appendix F.2). CLIMB's mean loss stays within its tolerance on every setting: at most 2.8 points at $\varepsilon = 5\%$ and 7.2 at 10%. At similar cost, always stopping at L_1 loses less on average than CLIMB at $\varepsilon = 10\%$ (3.3 against 5.0 points), but with no bound on any setting. At $\varepsilon = 5\%$ CLIMB's mean and largest losses are both below L_1's, though it spends 62% of tokens to L_1's 43%.

5.3 What a tolerance buys

The team improves on the direct answer on every setting, by 2.0 to 20.6 points, so savings are paid for in accuracy, and ε sets the price. At $\varepsilon = 2\%$ CLIMB saves little, at most 16% of the pipeline's tokens on gpt-oss-20b MuSiQue and nothing on MATH (Table 1, Figure 4a). At 5% it spends 31

Model	Benchmark	Pipeline	CLIMB $\varepsilon = 2\%$	CLIMB $\varepsilon = 5\%$	CLIMB $\varepsilon = 10\%$	Viol.
gpt-oss-20b	MATH	94.8 / 6.1k	94.7 / 100%	92.9 / 62%	89.6 / **33%**	0.0%
	FRAMES	76.1 / 93.9k	75.9 / 99%	73.6 / 66%	70.8 / **33%**	1.0%
	MuSiQue	64.9 / 15.0k	64.5 / 84%	63.0 / **44%**	58.1 / **31%**	1.0%
	BBEH	28.0 / 59.7k	27.4 / 98%	26.2 / 63%	24.7 / **22%**	2.0%
gpt-oss-120b	MATH	97.0 / 5.4k	96.9 / 100%	95.3 / **41%**	92.7 / **18%**	0.0%
	FRAMES	79.8 / 84.8k	79.6 / 97%	78.4 / 63%	75.1 / **32%**	1.0%
	MuSiQue	71.1 / 12.4k	70.4 / 91%	70.0 / **46%**	64.0 / **29%**	2.0%
	BBEH	45.1 / 53.3k	44.6 / 99%	42.8 / 75%	39.5 / **40%**	0.5%
Gemma-4-26B-A4B-it	MATH	99.8 / 10.4k	**99.8** / 100%	98.1 / **31%**	98.0 / **20%**	0.0%
	FRAMES	67.9 / 174.6k	**68.0** / 96%	66.4 / 86%	62.8 / 62%	1.0%
	MuSiQue	65.3 / 50.2k	**65.3** / 95%	62.7 / 69%	61.3 / **33%**	0.0%
	BBEH	36.4 / 120.2k	36.1 / 100%	33.5 / 96%	29.2 / 62%	1.5%

Table 1: Replay results: accuracy / tokens as a share of the pipeline's (pipeline: accuracy / mean tokens), averaged over 200 calibration/evaluation splits. CLIMB stops at L_0 on its log-probability or, depending on the depth chosen on the calibration half, at L_1 on agreement with L_0 and its log-probability. Viol. is the largest share of splits, over the three tolerances, in which the chosen policy's loss on the full recorded pool exceeds ε. Bold: under 50% of the pipeline's tokens, or at least as accurate as the pipeline.

Policy	Cert.	Tokens mean	Loss (points) mean	Loss (points) max
Always L_0		15%	8.7	20.6
Always L_1		43%	3.3	9.4
Threshold, same cost		61%	5.0	15.9
CLIMB, $\varepsilon = 5\%$	✓	62%	1.9	2.8
CLIMB, $\varepsilon = 10\%$	✓	35%	5.0	7.2
Oracle		65%	−7.2	0.0

Table 2: Uncertified rules against CLIMB over the twelve settings: tokens (% of the pipeline's) and accuracy lost (mean and largest). The threshold on L_0's confidence is fixed across settings to spend as much as CLIMB at $\varepsilon = 5\%$. The oracle stops at the cheapest correct rung and is not deployable.

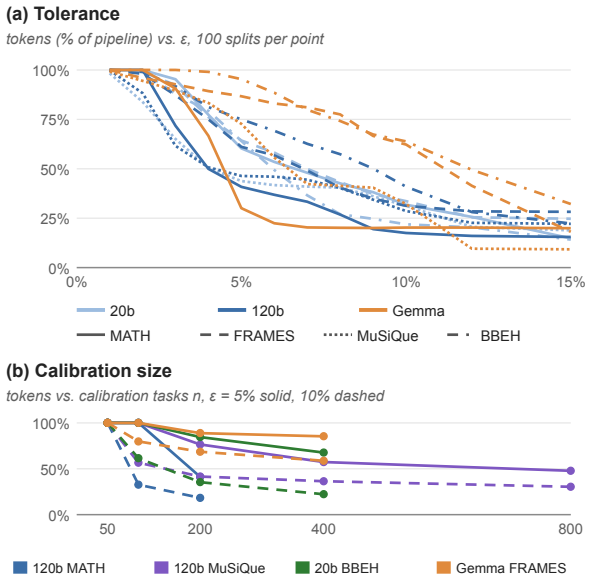


Figure 4: (a) Tokens as the tolerance ε varies (100 splits per point, violation rates at most 3%). (b) Tokens as the calibration set grows ($\varepsilon = 5\%$ and 10%) on four settings.

to 96% of the pipeline's tokens, 62% on average, for losses of at most 2.8 points. At 10% it spends 18 to 62%, 35% on average. Savings are smallest where the direct answer's confidence is inverted (Gemma on FRAMES and BBEH), where the certified policy largely escalates, and on FRAMES, whose long documents every rung pays for.

Savings also need data. With 50 calibration tasks CLIMB certifies almost nothing, with 100 only at 10%, and savings at 5% need a few hundred (Figure 4b). Recording them costs 1.5M to 79M tokens, repaid after 340 to 2,000 queries at $\varepsilon = 10\%$ and 460 to 4,400 at 5%, except on Gemma's BBEH, where the small saving at 5% takes 13,000 (Appendix F.6).

5.4 What limits the savings

Two things largely decide how much CLIMB can save, and neither affects the guarantee. *The signal.* By Proposition 1 a threshold is optimal when the gain from escalating falls with confidence. On the nine settings where L_0's confidence ranks answers correctly (AUROC 0.63 to 0.81, Figure 1c) the gain falls on eight: pooled over them, the gain falls from 21.6 points for the least confident tenth of tasks to 4 to 7 points in each of the five most confident tenths (Figure 5 in Appendix E.1). On Gemma's FRAMES, MuSiQue and BBEH the ranking is inverted (AUROC 0.22 to 0.34): greedy decoding without thinking falls into repetitions with high log-probability, and the certified policy

	$\varepsilon = 5\%$			$\varepsilon = 10\%$			
Variant	gpt	Gem.	All	gpt	Gem.	All	Viol.
Ref.: first draft	68	83	73	43	49	45	2%
Ref.: single agent L_1	94	80	89	77	49	67	3%
Random score	67	66	67	33	32	33	2%
Agreement only	70	95	79	46	76	56	3%
Log-prob. only	60	82	67	31	52	38	2%
Log-prob. + agr. (ours)	57	71	62	30	44	35	2%

Table 3: Ablations: mean tokens as a share of the reference's over gpt-oss's eight settings, Gemma's four and all twelve. The first two rows change the reference, with the deployed score. The rest change the score of the deployed ladder (random: mean of five draws), ending with CLIMB's. Viol.: largest violation rate over all settings and $\varepsilon \in \{2, 5, 10\}\%$. Per setting in Table 9.

relies on agreement or the team. *The design.* Stopping on the agreement of the two cheap rungs is cheaper where the direct answer is weak, a threshold on L_0 where it is strong. Choosing between them on calibration data, as CLIMB does, is among the cheapest of five designs certified the same way, and a learned estimate of the signed gain does no better on average (Appendix F.2). An oracle stopping at the cheapest correct rung beats the pipeline by 7.2 points on average (Table 2) and by up to 12, so much of the headroom lies in better signals.

5.5 Which score and which reference

Table 3 changes one ingredient at a time. *The score.* Every score keeps the guarantee, a random one included, but the score decides the savings. On gpt-oss, log-probability and agreement together save the most. On Gemma, whose direct answers are confidently wrong on three settings, adding agreement to log-probability spends less (71 against 82% at $\varepsilon = 5\%$), and a random score less still (66%): a random order meets the monotone-gain condition (M) trivially, while an inverted one front-loads the losses and stops the ordered test early. Where log-probability misleads, the certified policy falls back to the team, so a misleading score tends to cost savings rather than accuracy. At $\varepsilon = 10\%$, which exceeds the team's net gain over L_0 on most settings, even random stopping is affordable, and the signal matters most at tight tolerances (57 against 67% on gpt-oss at 5%). *The reference.* The guarantee holds for any reference, but the cheaper it is and the closer to the cheap rungs, the less to save: at $\varepsilon = 5\%$, CLIMB spends 73% of the reference's tokens with the first draft as reference, 89% with L_1 (Appendix F.1).

Test (ε=5%)	20b	120b	Gemma	Best
HB, signed in $[0, 1]$	100	100	100	0/12
Betting, clipped	82	75	80	0/12
Binomial, clipped	79	73	77	3/12
Betting, signed, Bonferroni	92	90	97	0/12
Betting, signed (ours)	**59**	**56**	**71**	**9/12**

Table 4: Tokens (% of the pipeline's, mean over the four benchmarks) certified at $\varepsilon = 5\%$ by tests of the same deployed family, and the number of settings where each is cheapest (ties count). HB: Hoeffding–Bentkus on the signed loss mapped to $[0, 1]$, as in Wynn et al. (2026). Clipped: the harm indicator $\max(\ell, 0)$. Per-setting values, with Hoeffding and clipped HB, in Table 7. Bold: best.

5.6 What makes certification efficient

Table 4 swaps parts of the procedure, keeping it valid. *Adapting to the variance matters.* On most stopped tasks the cheap answer and the reference are both right or both wrong, so most losses are zero. Mapped to $[0, 1]$ they sit at $1/2$, and the Hoeffding–Bentkus bound of Wynn et al. (2026), which assumes the worst variance at that mean, certifies nothing cheaper than the pipeline. *Keeping the sign also matters.* Clipping it to the harm indicator counts the answers stopping loses but not those it saves, and certifies much less where the team often loses correct answers, for instance 83 against 44% of the pipeline's tokens on gpt-oss-20b MuSiQue (Table 7). An exact binomial test of the clipped loss is cheapest on only three settings, two of them on MATH, where the team rarely loses correct answers. Bonferroni in place of sequential testing loses most of the rest.

6 Conclusion

A plan–execute–review team often costs several times a single agent, some tasks it solves do not need it, and it sometimes misses answers a single agent gets right. Whether to pay for it is a constrained decision with a signed loss, solved by a confidence threshold when escalation helps less the more confident the cheap answer is. CLIMB certifies that family with a variance-adaptive betting test whose bound holds, given i.i.d. data, whether or not the condition does. Uncertified rules of similar average cost can lose up to 16 points, while CLIMB's mean loss stayed within the tolerance on every setting, saving 38% of the pipeline's tokens on average at 5 points and 65% at 10. Where confidence misleads, it saves little rather than losing accuracy.

Limitations

Scope of the guarantee. As with other distribution-free risk-control methods, the guarantee assumes that calibration and deployment tasks, and the model's outputs on them, share one distribution, so thresholds are best calibrated on the target tasks and serving stack (Appendix E.2). Once records exist, recalibration takes under a second (Appendix C), and weighted or online calibration could extend the guarantee to shifting distributions.

Calibration data. CLIMB needs every rung's outcome on a few hundred calibration tasks, a one-off cost that is typically repaid within a few thousand queries (§5.3). With less data or tighter tolerances it falls back to the full pipeline, trading savings for the guarantee. Stronger confidence signals could plug in directly and raise the savings.

References

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. 2024. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems*.

Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. 2025. Learn then test: Calibrating predictive algorithms to achieve risk control. *The Annals of Applied Statistics*, 19(2):1641–1662.

Anastasios N. Angelopoulos, Stephen Bates, Adam Fisch, Lihua Lei, and Tal Schuster. 2024. Conformal risk control. In *International Conference on Learning Representations*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*.

Weize Chen, Jiarui Yuan, Chen Qian, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2025. Optima: Optimizing effectiveness and efficiency for LLM-based multi-agent system. In *Findings of the Association for Computational Linguistics: ACL 2025*.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. 2024. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations*.

Yifan Dou, Shikan Lian, and Shibo Li. 2026. Conformal Cascade: Distribution-free accuracy guarantees for multi-tier LLM inference. *arXiv preprint arXiv:2607.25018*.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning*.

Sugyeong Eo, Hyeonseok Moon, Evelyn Hayoon Zi, Chanjun Park, and Heuiseok Lim. 2025. Debate only when necessary: Adaptive multiagent collaboration for efficient LLM reasoning. *arXiv preprint arXiv:2504.05047*.

Gemma Team. 2026. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*.

May Hamri and Inbal Talgam-Cohen. 2026. ZEBRA: Zero-shot budgeted resource allocation for LLM orchestration. *arXiv preprint arXiv:2605.20485*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Metod Jazbec, Alexander Timans, Tin Hadži Veljković, Kaspar Sakmann, Dan Zhang, Christian A. Naesseth, and Eric Nalisnick. 2024. Fast yet safe: Early-exiting with risk control. In *Advances in Neural Information Processing Systems*.

Jaehun Jung, Faeze Brahman, and Yejin Choi. 2025. Trust or escalate: LLM judges with provable guarantees for human agreement. In *International Conference on Learning Representations*.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. 2022. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Mehran Kazemi, Bahare Fatemi, Hritik Bansal, John Palowitch, Chrysovalantis Anastasiou, Sanket Vaibhav Mehta, Lalit K. Jain, Virginia Aglietti, Disha Jindal, Peter Chen, Nishanth Dikkala, Gladys Tyen, Xin Liu, Uri Shalit, Silvia Chiappa, Kate Olszewska, Yi Tay, Vinh Q. Tran, Quoc V. Le, and Orhan Firat. 2025. BIG-bench extra hard. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26473–26501.

Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqui. 2025. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics*.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations*.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems*.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *International Conference on Learning Representations*.

Tengxiao Liu, Zifeng Wang, Jin Miao, I-Hung Hsu, Jun Yan, Jiefeng Chen, Rujun Han, Fangyuan Xu, Yanfei Chen, Ke Jiang, Samira Daruki, Yi Liang, et al. 2025. Budget-aware tool use enables effective agent scaling. *arXiv preprint arXiv:2511.17006*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.

Christopher Mohri and Tatsunori Hashimoto. 2024. Language models with conformal factuality guarantees. In *Proceedings of the 41st International Conference on Machine Learning*.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to route LLMs from preference data. In *International Conference on Learning Representations*.

OpenAI. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.

Edoardo Pona, Milad Kazemi, Mehran Hosseini, Yali Du, David Watson, Osvaldo Simeone, and Nicola Paoletti. 2026. Calibrate-then-delegate: Safety monitoring with risk and budget guarantees via model cascades. *arXiv preprint arXiv:2604.14251*.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Victor Quach, Adam Fisch, Tal Schuster, Adam Yala, Jae Ho Sohn, Tommi S. Jaakkola, and Regina Barzilay. 2024. Conformal language modeling. In *International Conference on Learning Representations*.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*.

Andries Petrus Smit, Nathan Grinsztajn, Paul Duckworth, Thomas D. Barrett, and Arnu Pretorius. 2024. Should we be going MAD? A look at multi-agent debate strategies for LLMs. In *Proceedings of the 41st International Conference on Machine Learning*, pages 45883–45905.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.

Jean Ville. 1939. *Étude Critique de la Notion de Collectif*. Gauthier-Villars.

Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. *Algorithmic Learning in a Random World*. Springer.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*.

Zhexuan Wang, Yutong Wang, Xuebo Liu, Liang Ding, Miao Zhang, Jie Liu, and Min Zhang. 2025. AgentDropout: Dynamic agent elimination for token-efficient and high-performance LLM-based multi-agent collaboration. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Zheyuan Wang, Siyu Li, Peiqiao Song, Sijia Chen, Qianqian Song, and Qian Liu. 2026. Signed Rescue Routing: Harm-aware cascades for efficient LLM inference. *arXiv preprint arXiv:2609.07786*.

Ian Waudby-Smith and Aaditya Ramdas. 2024. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(1):1–27.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *Conference on Language Modeling*.

Andrea Wynn, Metod Jazbec, Charith Peris, Rinat Khaziev, Anqi Liu, Daniel Khashabi, and Eric Nalisnick. 2026. Controlling the risk of corrupted contexts for language models via early-exiting. In *Proceedings of the 43rd International Conference on Machine Learning*.

Andrea Wynn, Harsh Satija, and Gillian Hadfield. 2025. Talk isn't always cheap: Understanding failure modes in multi-agent debate. *arXiv preprint arXiv:2509.05396*.

Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *International Conference on Learning Representations*.

Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. 2024. Large language model cascades with mixture of thought representations for cost-efficient reasoning. In *International Conference on Learning Representations*.

Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. 2025. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. In *International Conference on Learning Representations*.

A Proofs

Answers are binary and costs finite (every call has a generation cap and the pipeline at most five rounds), so all expectations exist. Propositions 1 and 3 concern the one-decision setting of §3.2, with cheap answer correctness Y_c, cost C_c and score S. For a policy $\pi : s \mapsto \pi(s) \in [0,1]$ other than $\varnothing$, $J(\pi) = \mathbb{E}[C_c] + \mathbb{E}[C_{\text{ref}}] - \mathbb{E}[\pi(S)\, c(S)]$ and $R(\pi) = \mathbb{E}[\pi(S)\, g(S)]$ by the tower property, with g and c as in Eq. (3).

Proposition 1. Among policies other than $\varnothing$, minimising J subject to $R \le \varepsilon$ is maximising $\mathbb{E}[\pi(S)\, c(S)]$ subject to $\mathbb{E}[\pi(S)\, g(S)] \le \varepsilon$ over measurable π with values in $[0,1]$: a linear program with one constraint, strictly feasible because $\pi \equiv 0$ has $R = 0 < \varepsilon$. By Slater's condition there is a multiplier $\mu^* \ge 0$ such that every solution maximises $\mathbb{E}[\pi(S)\,(c(S) - \mu^* g(S))]$ and satisfies complementary slackness. If $\mu^* = 0$ the constraint is slack and $\pi \equiv 1$, the threshold $t = -\infty$, is optimal. Otherwise the maximiser sets $\pi(s) = 1$ where $g(s)/c(s) < 1/\mu^*$ and $\pi(s) = 0$ where $g(s)/c(s) > 1/\mu^*$, using $c > 0$. By (M) the first set is an upper interval of scores and the second a lower one, and the set where $g/c = 1/\mu^*$ is an interval between them. On it any π is optimal for the Lagrangian, and complementary slackness fixes only the total risk $\mathbb{E}[\pi(S)\, g(S)] = \varepsilon$. Stopping on the upper part $\{S > t'\}$ of that interval, with t' chosen so that the total risk equals ε, is possible exactly when $u \mapsto \mathbb{E}[g(S)\,\mathbf{1}[S > u]]$ attains ε, which holds when S has no atom at t', in particular when its distribution is continuous. Otherwise the policy stops on the atom with the probability that makes the risk equal ε. The result is a threshold policy, randomised at most on one atom. Comparing its cost with $J(\varnothing)$ gives the optimum over all policies.

CLIMB searches a finite grid of deterministic thresholds. With a continuous score, the only gap to this optimum is the grid's resolution, and Appendix F.5 gives an example with an atom, where randomising helps.

Proposition 2. *Validity.* Fix m and suppose $\mathcal{H}_m$ holds, so the losses $\ell_1, \ell_2, \dots$ of threshold t_m on i.i.d. calibration tasks have mean $\mu > \varepsilon$. Each λ_i is a function of $\ell_1, \dots, \ell_{i-1}$ with $0 \le \lambda_i < 1/(1-\varepsilon)$, so $1 + \lambda_i(\varepsilon - \ell_i) > 0$ since $\ell_i \le 1$, and $\mathbb{E}[1 + \lambda_i(\varepsilon - \ell_i) \mid \ell_{1:i-1}] = 1 + \lambda_i(\varepsilon - \mu) \le 1$. Hence W_j is a nonnegative supermartingale with $W_0 = 1$, and

by Ville's inequality $\Pr[\max_j W_j \geq 1/\alpha] \leq \alpha$, i.e. $\Pr[p_m \leq \alpha] \leq \alpha$, at the level $\alpha = \delta/2$ of each family. *Family-wise error.* Let m^* be the first index at which $\mathcal{H}_m$ is true, which depends only on the distribution. The procedure rejects $\mathcal{H}_m$ only after rejecting every earlier hypothesis, so any false rejection requires rejecting $\mathcal{H}_{m^*}$, which has probability at most α. *Risk.* On the complementary event every threshold in $\mathcal{C}$ has $\mathbb{E}[\ell] \leq \varepsilon$, and $\varnothing$ has $\ell \equiv 0$, so $R \leq \varepsilon$ for every member of $\mathcal{C}$. By a union bound over the two families, this holds for every member of $\mathcal{C}_0 \cup \mathcal{C}_1$ with probability at least $1 - \delta$, however the deployed one is selected. *Cost.* The deployed policy minimises mean calibration cost over $\mathcal{C} \ni \varnothing$.

The i.i.d. assumption is used in the conditional expectation above: each calibration loss must be a fresh draw from the deployment distribution. If the losses share a component that varies from one run of the model to the next, the bound holds for the recorded run but not necessarily for a new one.

Proposition 3. *(i)* Let $q(t) = \Pr[S > t]$ be the share of tasks threshold t stops, and write the risk as a function of it, $r = \int_0^q \tilde{g}(v)\,dv$, where $\tilde{g}(v) = g(F^{-1}(1-v))$ is the gain of the task at upper quantile v of the score distribution F (with atoms, r is the piecewise-linear interpolation of the same sums). Moving to a lower threshold raises q and adds tasks of lower score, whose gain is at least as large by (M), so $\tilde{g}$ is non-decreasing and r is convex in q, with $r(0) = 0 \leq \varepsilon$. The set $\{q : r(q) \leq \varepsilon\}$ is therefore an interval $[0, q_\varepsilon]$, and since q increases along the candidate sequence, the thresholds with $r \leq \varepsilon$ are exactly its first m_ε members. *(ii)* $J(t) = \mathbb{E}[C_c] + \mathbb{E}[C_{\text{ref}}\,\mathbf{1}[S \leq t]]$ does not increase as t decreases, since $C_{\text{ref}} \geq 0$. *(iii)* Fix $\eta > 0$ and let $m' = m_{\varepsilon-\eta}$. For $m \leq m'$ the losses of t_m have mean at most $\varepsilon - \eta$. The running variance estimate converges, so the bets λ_i are eventually of order $n^{-1/2}$ and bounded so that $\lambda_i(\varepsilon - \ell_i) \geq -\frac{1}{2}$. With $\log(1+x) \geq x - x^2$ for $x \geq -\frac{1}{2}$, $\log W_n \geq \sum_i \lambda_i(\varepsilon - \ell_i) - \sum_i \lambda_i^2(\varepsilon - \ell_i)^2$, whose first sum grows as $\eta\sqrt{n}$ up to a constant and whose second stays bounded. Hence $p_m \to 0$ in probability for each of the finitely many $m \leq m'$, and with probability tending to one all of $t_1, \ldots, t_{m'}$ are certified. On that event the deployed threshold has mean calibration cost at most that of $t_{m'}$, and since mean calibration costs converge to J uniformly over the finite grid, its expected cost is at most $J(t_{m'}) + o_P(1)$. Letting $\eta \to 0$ on a refining grid, $J(t_{m_{\varepsilon-\eta}})$ tends to the cost of the best threshold at tolerance ε when r is continuous, which by Proposition 1 is the optimal cost.

B Experimental Details

Benchmarks and splits. Table 5 lists the benchmarks. MATH, FRAMES and MuSiQue were split once at random (seed 42) into a development split (20%), used while building the pipeline and its prompts, and a test split (80%), used for every reported number. For BBEH (Kazemi et al., 2025) we draw 38 problems from each of its 23 tasks with a seed fixed per task. Nothing was tuned on them.

Documents. MuSiQue's twenty paragraphs (median 2.4k tokens) are given in the dataset's order, each under its title. For FRAMES we fetch the wikitext of the two to nineteen linked articles (median 17k tokens) from the Wikipedia API in October 2026 and render it to text, keeping the infoboxes and tables that some questions need and that public plain-text dumps omit. Where a question's articles exceed 80k tokens (6 of 660 questions), each keeps its head up to an equal share of that limit.

Scoring. *MATH* answers, taken from the last `ANSWER:` line or boxed expression, are compared with the reference by symbolic equivalence (math-verify) after common plain-text notations are normalised. *BBEH* uses the fuzzy match of its official evaluation. *FRAMES and MuSiQue* are scored by an LLM judge, since exact match marks correct but verbose answers wrong: gpt-oss-120b at medium reasoning effort and temperature 0 receives the question, the reference (with MuSiQue aliases) and the extracted answer, and replies CORRECT only if the answer gives the same entity, number, date or range. Answers already correct under exact match are accepted without a call. The judge accepted all of 60 answers correct under exact match and agreed with itself on 99.5% of a sample judged twice.

Models and tool. gpt-oss-20b and gpt-oss-120b use their released MXFP4 weights, with low reasoning effort at L_0 and medium effort elsewhere. Gemma-4-26B-A4B-it is the released standard checkpoint, decoded greedily at L_0 and sampled elsewhere at temperature 1.0, top-p 0.95 and top-k 64, as its model card recommends. Python runs in a separate process with a 10-second limit, without

Benchmark	Split	Pool	Used
MATH-500	test	500	400
FRAMES	test	824	660
MuSiQue	validation, answerable	2,417	1,934
BBEH	23 tasks, 38 each	4,520	874

Table 5: Benchmarks. FRAMES and MuSiQue are open-book (documents in every prompt).

Algorithm 1 CLIMB calibration and deployment

Require: calibration tasks with recorded Y_k, C_k, S_k, Y_{ref}, C_{ref}; ε, δ; grid $t_1 > \cdots > t_M$

for depth $D = 0, 1$ **do** ▷ decide after L_0; after L_0 and L_1
  $\mathcal{C}_D \leftarrow \{\varnothing\}$
  for $m = 1, \ldots, M$ **do**
    $\ell_i \leftarrow Y_{\mathrm{ref}}(x_i) - Y_{\pi_{D,t_m}}(x_i)$ for $i = 1, \ldots, n$
    if $1/\max_j W_j(\ell_{1:j}) \le \delta/2$ **then** $\mathcal{C}_D \leftarrow \mathcal{C}_D \cup \{t_m\}$
    else break
$(D^\star, t^\star) \leftarrow$ the member of $\mathcal{C}_0 \cup \mathcal{C}_1$ with the lowest mean calibration cost
Deploy: if $t^\star = \varnothing$ run the pipeline; else run $L_0, \ldots, L_{D^\star}$ and return $L_{D^\star}$'s answer if $S_{D^\star} > t^\star$; otherwise run the pipeline.

system, file or `eval` access, and its output is truncated to 1,000 characters. A call may execute code up to three times. Calls that failed were retried.

Budget and cost. There is no per-task budget. Every call is charged its prompt and completion tokens, including hidden reasoning, and a task's cost includes its tool and follow-up calls. Generation is capped per call at 8,192 tokens at L_0 and at 4,096 (gpt-oss on MATH, FRAMES and MuSiQue) or 16,384 tokens elsewhere. A reply that ends without an answer receives one closing call at the lowest reasoning setting. Pipeline runs that still end without an answer (at most 0.5% of tasks) count as errors.

Reviewer verdict. The verdict is read from the reviewer's last `VERDICT:` line, or failing that from a leading yes or no or its wording. A review left empty by the generation cap receives one closing call that asks for the verdict alone.

C Implementation of CLIMB and the Replay Protocol

Betting test. For a candidate threshold the signed losses $\ell_1, \ldots, \ell_n$ on the calibration tasks are processed in a random order. With $s_1 = \sum_{i<j} \ell_i$ and $s_2 = \sum_{i<j} \ell_i^2$ over the earlier tasks, the bet on task j is $\lambda_j = \min\big(\frac{1}{2(1-\varepsilon)}, \sqrt{2\log(1/\delta)/(n\,\hat\sigma_j^2)}\big)$ with $\hat\sigma_j^2 = \max\big((s_2 + 0.25)/j - (s_1/j)^2,\ 10^{-3}\big)$, a variance estimate from earlier tasks shrunk towards 0.25. The p-value is $1/\max_j W_j$.

Grid and selection. CLIMB certifies two one-decision families, each at level $\delta/2$. The first stops at L_0 when $S_0 > t$, and the second runs both rungs and stops at L_1 when $S_1 = 10 \cdot \mathbf{1}[a_0 = a_1] + \bar s_1 > t$. Candidates are $\varnothing$ (always run the pipeline), then thresholds from conservative to aggressive: for S_0, $-0.005, -0.010, \ldots, -0.600$, then $-0.65, -0.70, \ldots, -3.0$, then $-\infty$. For S_1 it is the same grid shifted by 10, which stops agreeing tasks first, followed by the unshifted grid. Each family is tested in order with the betting test, and among the certified candidates of both families the one with the lowest mean calibration cost is deployed. A rung without log-probabilities does not stop. For agreement, the text after the last answer line (matched case-insensitively, with or without markdown: `ANSWER:`, `Answer:`, `**Answer:**`) is normalised as in SQuAD (lower case, no punctuation or articles, collapsed whitespace) with Unicode spaces mapped to ASCII. Calibration takes under a second on 1,000 tasks.

Evaluation. Records and replay are as in §5.1. Each reported number averages 200 random calibration/evaluation splits (seed fixed per analysis), except the sweeps over ε, δ and the calibration size, the comparisons of tests (Tables 4 and 7) and of policies, which use 100 splits.

Richer policies. The policies compared in Appendix F.2 use a training split (20%), are certified with the same betting test on a calibration split (40%) and evaluated on the rest. (A) is the first family alone and (B) a shared threshold on the two rungs' mean log-probabilities, stopping at L_0 when $S_0 > t$ and otherwise at L_1 when its mean log-probability exceeds t. (C) is the second family alone and (D) the deployed choice between (A) and (C) at $\delta/2$ each. (E) ranks tasks by a learned estimate of the signed gain of escalating instead of stopping at L_1, $P(\text{rescue}) - P(\text{harm})$, from a gradient-boosted classifier fitted on the training split to the two scores, their agreement and the two rungs' costs. Its thresholds are quantiles of the training scores, fixed before calibration.

D Checking the Guarantee under i.i.d. Calibration

The violation rates of the main experiments evaluate the selected policy on the recorded pool from

Model	ε	Worst setting	$\Pr(R > \varepsilon)$ [95% CI]
gpt-oss-20b	2%	MuSiQue	3.40 [2.9, 3.9]
	5%	FRAMES	2.02 [1.6, 2.4]
	10%	BBEH	2.40 [2.0, 2.9]
gpt-oss-120b	2%	BBEH	2.18 [1.8, 2.6]
	5%	FRAMES	2.34 [1.9, 2.8]
	10%	BBEH	0.22 [0.1, 0.4]
Gemma	2%	MuSiQue	3.08 [2.6, 3.6]
	5%	BBEH	3.32 [2.8, 3.9]
	10%	BBEH	1.88 [1.5, 2.3]

Table 6: Direct check of Proposition 2: the worst empirical failure probability across the four benchmarks for each model and tolerance. Each estimate uses 5,000 i.i.d. calibration draws from the recorded pool. All upper 95% limits remain below $\delta = 10\%$.

which its calibration half was drawn without replacement, a finite-pool diagnostic rather than the event bounded by Proposition 2. To check the proposition itself we take the task distribution $\mathcal{D}$ to be the recorded pool of each setting, uniform over its tasks. We draw a calibration set of $n = \lfloor N/2 \rfloor$ tasks i.i.d. from $\mathcal{D}$ (with replacement), calibrate CLIMB as deployed, and compute the risk $R(\hat{\pi})$ of the selected policy under $\mathcal{D}$ exactly, as its mean loss over the pool, which involves no evaluation sample. Repeating 5,000 times estimates $\Pr(R(\hat{\pi}) > \varepsilon)$, which the proposition bounds by $\delta = 10\%$. Table 6 reports it with Clopper–Pearson intervals. The largest estimate is 3.40% (gpt-oss-20b MuSiQue, $\varepsilon = 2\%$), and every upper 95% limit is at most 3.9%. The failure probability is highest where the cheapest certified threshold sits close to the boundary $R = \varepsilon$, and zero where the certified family stops well inside it: at $\varepsilon = 10\%$ it is zero on every setting except BBEH, where it is at most 2.4%.

E Additional Diagnostics

E.1 Monotone-gain test

Figure 6 shows, per setting, the gain of escalating instead of stopping at L_0 by quintile of L_0's confidence, the quantity of condition (M). It falls with confidence on eight of the twelve settings (Spearman ρ from -0.3 to -1.0), is flat on gpt-oss-20b BBEH ($\rho = 0.0$) and rises on Gemma FRAMES, MuSiQue and BBEH ($\rho = 0.87$, 0.70 and 0.70). There L_0's confidence is inverted (AUROC 0.30, 0.34 and 0.22): greedy decoding without thinking loops into repetitions that receive a high log-probability. Pooled over all twelve settings, rather than the nine of Figure 5, the gain is

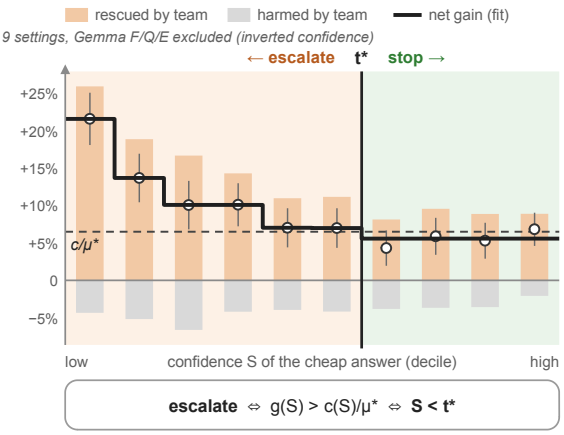


Figure 5: Share of tasks the pipeline rescues (up) and harms (down) by decile of L_0's confidence, pooled over the nine settings where that confidence ranks answers correctly. The net gain falls with confidence, so stopping pays above one threshold $t^\star$.

therefore U-shaped, from 14.2 points for the least confident tenth of tasks down to 4.8 and up to 16.9 for the most confident, the last driven by Gemma. The guarantee does not depend on the condition. Where it fails, the threshold on S_0 certifies little and the agreement family carries the savings.

E.2 Sensitivity and distribution shift

Table 8 varies the error level and the calibration size, and tests transfer across benchmarks. A larger δ buys modest savings, and savings at $\varepsilon = 5\%$ require a few hundred calibration tasks: with 50 or 100, CLIMB falls back to the pipeline almost everywhere. A policy certified on one benchmark and deployed on another of the same model loses up to 16.0 points (Gemma, MATH to BBEH) and more than 5 in ten of 36 cases: the guarantee need not transfer across distributions. At a tolerance of $\varepsilon = 1\%$, CLIMB certifies nothing on nine settings and spends 98 to 99.8% of the pipeline's tokens on the three MuSiQue settings (100 splits), with no violation.

F Further Analyses

All analyses in this section replay the per-task records released with the paper. Intervals over random splits reflect split randomness only, not independent re-runs of the benchmarks.

F.1 Ablations of the score and the reference

Table 9 changes one part of the deployed procedure at a time and replays it on the splits of Table 1.

Test (ε=5%)	20b				120b				Gemma			
	M	F	Q	E	M	F	Q	E	M	F	Q	E
Hoeffding–Bentkus, signed loss in $[0, 1]$	100	100	100	100	100	100	100	100	100	100	100	100
Hoeffding, signed loss in $[0, 1]$	100	100	100	100	100	100	100	100	100	100	100	100
Betting, clipped loss	71	85	83	89	43	90	75	89	34	95	96	95
Hoeffding–Bentkus, clipped loss	73	84	84	90	44	92	76	91	36	95	95	97
Binomial, clipped loss	65	80	82	87	**40**	88	74	88	**25**	94	94	**93**
Betting, signed loss, Bonferroni	100	98	73	96	100	97	67	97	100	98	91	100
Betting, signed loss (ours)	**63**	**65**	**44**	**66**	41	**61**	**47**	**76**	34	**86**	**71**	94

Table 7: Tokens (% of the pipeline's) certified at $\varepsilon = 5\%$ by different tests of the same deployed family, over 100 splits. The signed loss $\ell \in \{-1, 0, 1\}$ is mapped to $(\ell + 1)/2$ with the tolerance mapped to $(\varepsilon + 1)/2$ for the bounded-range tests, as in Wynn et al. (2026). The clipped loss $\max(\ell, 0)$ is the harm indicator. All tests are fixed-sequence except Bonferroni, which divides δ by the number of candidates. Bold: cheapest. M/F/Q/E: MATH/FRAMES/MuSiQue/BBEH.

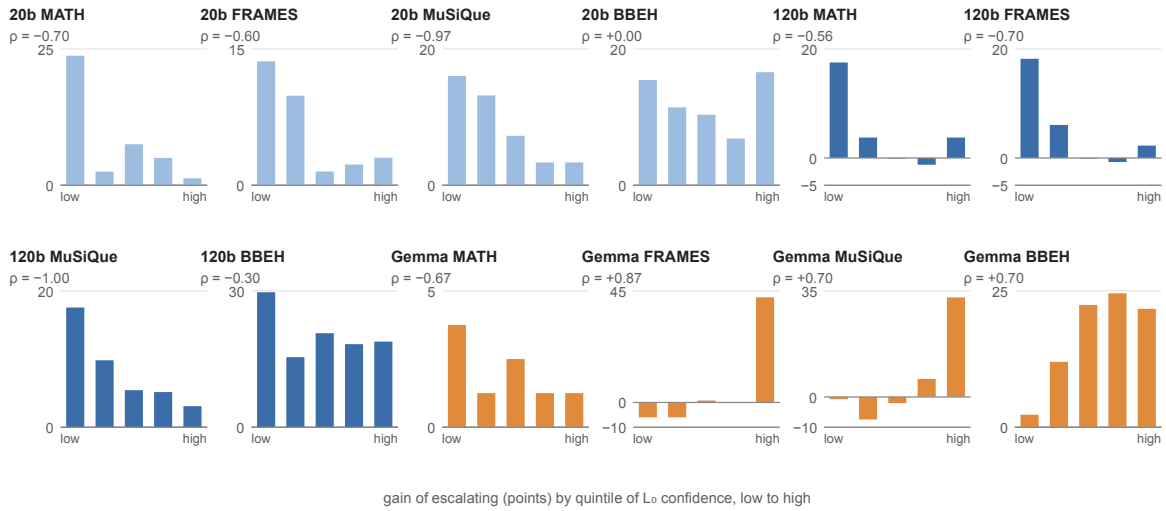


Figure 6: Gain of escalating instead of stopping at L_0, by quintile of L_0's confidence (Q1 least confident), on twelve settings. Inset: Spearman ρ, which is non-positive on nine settings and positive on Gemma FRAMES, MuSiQue and BBEH, where L_0's confidence is inverted.

	20b				120b				Gemma			
	M	F	Q	E	M	F	Q	E	M	F	Q	E
(a) Tokens (% of pipeline) at $\varepsilon = 5\%$ as δ varies (n = half the pool)												
$\delta = 0.05$	71	70	45	70	44	69	47	78	47	89	75	97
$\delta = 0.1$	61	65	44	63	40	61	47	74	32	86	69	94
$\delta = 0.2$	52	61	42	57	38	56	47	72	23	85	65	92
(b) Tokens (% of pipeline) at $\varepsilon = 5\%$ as the calibration set grows ($\delta = 0.1$)												
$n = 50$	100	100	100	100	100	100	100	100	100	100	100	100
$n = 100$	100	100	100	100	100	100	100	100	100	100	100	100
$n = 200$	61	79	75	84	41	75	76	87	30	89	90	97
$n = 400$	–	62	58	68	–	60	57	76	–	85	83	95
$n = 800$	–	–	46	–	–	–	48	–	–	–	75	–
(c) Loss (points) of the policy certified at $\varepsilon = 5\%$ on one benchmark (row), deployed on another (column)												
from MATH	–	+0.2	−0.3	+1.3	–	+0.6	+1.0	**+7.0**	–	**+7.4**	**+7.2**	**+16.0**
from FRAMES	**+5.3**	–	+2.9	+4.5	+0.8	–	+1.1	**+7.1**	+0.3	–	+1.6	**+5.5**
from MuSiQue	+3.3	+2.7	–	+3.2	+0.8	+0.6	–	**+7.1**	+1.3	**+5.3**	–	**+9.4**
from BBEH	+2.8	+2.3	+1.1	–	0.0	0.0	+0.7	–	0.0	+0.9	+0.9	–

Table 8: Robustness diagnostics. (a) and (b): means over 100 splits. In (b), –: a calibration set larger than 80% of the pool. (c): the policy is certified on the full pool of one benchmark and deployed on another with the same model. Bold: a loss above the 5-point tolerance. M/F/Q/E: MATH/FRAMES/MuSiQue/BBEH.

	20b				120b				Gemma					
Variant	M	F	Q	E	M	F	Q	E	M	F	Q	E	Mean	Viol.
$\varepsilon = 5\%$														
Reference: the team's first draft	61	85	64	91	33	65	70	72	38	100	97	97	73	2.0%
Reference: the single agent L_1	79	99	99	93	80	99	100	100	26	100	100	96	89	3.0%
Random score	81	79	51	72	49	72	47	86	42	**78**	**54**	**90**	67	2.1%
Agreement only	80	89	57	89	44	71	47	87	99	99	83	100	79	3.0%
Log-prob. only	72	67	**44**	**54**	**41**	64	**46**	91	**31**	100	99	97	67	2.0%
Log-prob. and agreement (CLIMB)	**62**	**66**	**44**	63	**41**	**63**	**46**	**75**	**31**	86	69	96	**62**	2.0%
$\varepsilon = 10\%$														
Reference: the team's first draft	27	50	35	65	21	46	38	66	28	69	36	65	45	2.0%
Reference: the single agent L_1	51	94	83	56	61	89	92	88	26	80	35	53	67	3.0%
Random score	36	35	33	23	**18**	36	37	47	**19**	**34**	**12**	**61**	**33**	2.1%
Agreement only	40	51	41	26	38	58	46	69	99	70	41	95	56	3.0%
Log-prob. only	**33**	**33**	**31**	**22**	**18**	**32**	**29**	48	20	91	33	65	38	2.0%
Log-prob. and agreement (CLIMB)	**33**	**33**	**31**	**22**	**18**	**32**	**29**	**40**	20	62	33	62	35	2.0%

Table 9: Ablations of the score and of the reference, replayed on the same 200 splits as Table 1: tokens as a share of the reference's, at $\varepsilon = 5\%$ and 10%. The first two rows change the reference that the loss is measured against, with the deployed score. The other four change only the score of the deployed ladder (the random score is drawn independently of everything, mean of five draws), ending with CLIMB's. The first draft's cost is approximate (planner tokens plus the executor's divided by the number of rounds). Viol.: largest violation rate over the twelve settings and the three tolerances 2, 5 and 10%. Bold: cheapest score. M/F/Q/E: MATH/FRAMES/MuSiQue/BBEH.

The score. The guarantee holds for every score, including a random one: the largest violation rate is 3% against $\delta = 10\%$. The score decides how much is saved. At $\varepsilon = 5\%$ log-probability and agreement together save more than either alone and more than a random score: 62% of the pipeline's tokens on average, against 67% for log-probability only, 79% for agreement only and 67% for a random score. Adding agreement matters most where the direct answer's confidence is inverted: on Gemma's FRAMES and MuSiQue, log-probability alone certifies almost nothing (100 and 99%), while the deployed score saves 14 and 31%. At 10 points, where the team's net gain is small next to the tolerance, a random score spends about as much on average (33 against 35%): slightly more than the deployed score on gpt-oss, by up to 8 points, and less on Gemma's FRAMES and MuSiQue, where log-probability ranks answers backwards. Either way the guarantee holds: a misleading score costs savings rather than accuracy.

The reference. The loss can be measured against any reference. The team's first draft, written by the executor from the plan before any review, is within 2 points of L_1 on ten of twelve settings. Measured against it, CLIMB stays within the tolerance and spends 73% of its tokens on average at $\varepsilon = 5\%$ and 45% at 10%. Against L_1 itself, with the ladder $L_0 \to L_1$, it spends 89 and 67%: the cheaper the reference and the closer it is to the cheap rungs, the less there is to save.

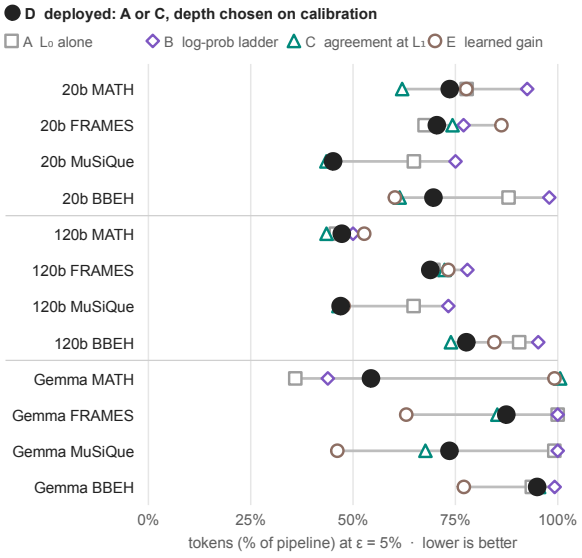


Figure 7: Tokens at $\varepsilon = 5\%$ for five escalation designs certified at the same δ (20/40/40 splits): (A) a threshold on L_0's confidence, (B) one log-probability threshold shared by L_0 and L_1, (C) stopping at L_1 on its agreement with L_0, (D) CLIMB, which chooses between A and C on calibration, and (E) a learned estimate of the signed gain.

F.2 Escalation designs under one guarantee

Because CLIMB certifies any ordered family, richer escalation designs can be compared with the deployed one on equal terms: each is held to the same tolerance, and only its cost differs. Read as an ablation, A drops the second rung, B drops

	Violation rate (%)				Tokens (%)	
	Tuned			CLIMB		
ε	mean	max	$> \delta$	max	Tuned	CLIMB
2%	38.3	66.5	10/12	1.0	57	97
5%	30.8	61.0	9/12	2.0	37	62
10%	4.3	48.5	1/12	2.0	20	35

Table 10: A threshold tuned per setting on the calibration half without certification (Tuned), against CLIMB, on the 200 splits of Table 1. Violation rate, as in Table 1: share of splits in which the deployed policy's loss on the full recorded pool exceeds ε, mean and maximum over the twelve settings. $> \delta$: settings above $\delta = 10\%$. Tokens: mean share of the pipeline's.

agreement, and C drops the stop at L_0.

Agreement and depth. The agreement of the two cheap rungs is a strong signal where both answer in a short form: on tasks where L_0 and L_1 give the same answer, L_1 is right 79 to 100% of the time on MATH, FRAMES and MuSiQue, against 34 to 86% when they disagree. On BBEH, whose answers are long and varied, the two agree on only 24 to 41% of tasks, and agreement says little for gpt-oss-20b (26 against 25%) but much for Gemma (58 against 17%). Stopping on it (C) is often the cheapest design where the direct answer is weak, for instance 43% of the pipeline's tokens on gpt-oss-20b MuSiQue against 65% for a threshold on L_0 alone (A), and it still saves where L_0's confidence is inverted (Gemma on FRAMES and MuSiQue, where A certifies almost nothing). Where the direct answer is already strong, as for Gemma on MATH, C pays for L_1 on every task and A is far cheaper (36% against 101%). Choosing between them on calibration data (D, CLIMB) avoids both failures and is among the cheapest on average, 68% of the pipeline's tokens against 69 to 82% for A, B and C (Figure 7). A learned estimate of the signed gain (E) matches it on average. It saves more where it can learn that the confidence is inverted (Gemma FRAMES, 63% against 87%) and less elsewhere (gpt-oss-20b FRAMES, 86% against 70%), and it needs a training split.

Tuning without certification. The threshold of Table 2 is shared across settings. A stronger uncertified baseline tunes it per setting on the same calibration half: it deploys the cheapest candidate of either family whose empirical loss there is at most ε, with CLIMB's score, candidates and splits. It is cheaper but often exceeds the tolerance (Table 10). At $\varepsilon = 2\%$ and 5%, its violation rate averages 38 and 31% and exceeds δ on ten and nine of the twelve settings, against at most 2% for CLIMB. A threshold tuned to the edge of the tolerance on one sample tends to cross it on the next. At $\varepsilon = 10\%$ it stays within δ on eleven settings, where the cheapest candidates are already safe. The tokens CLIMB spends beyond it are thus largely the price of the guarantee.

F.3 What the team adds, by setting

Figure 9 shows why the team's first-draft errors survive review: on every setting the reviewer accepts most of the executor's correct answers, but also 23 to 58% of its incorrect ones, so a wrong plan-guided draft is often passed through rather than revised.

F.4 Seeding the team with the cheap answer

An escalated task runs the reference from scratch. Handing the team L_1's answer and asking it to verify it first did not help in our check: on gpt-oss-20b MATH the seeded pipeline answers 94.0% of the tasks against 94.8% for the reference, and on the 49 tasks L_1 gets wrong, the ones escalation is for, 71.4% against 73.5%. Running the reference from scratch also makes the escalated outcome equal to the reference's, so the loss comes only from stopped tasks.

F.5 Randomised policies

Proposition 1 allows the constrained optimum to randomise when the score has an atom. In the two-level example with a single score, a cheap rung that is always wrong at cost 1 and a top rung that is always right at cumulative cost 2, any deterministic policy with risk at most $\varepsilon = 5\%$ must always escalate (cost 2), whereas escalating with probability 0.95 costs 1.95. With near-continuous scores, as here, the gap is at most the grid's resolution, and CLIMB deploys deterministic thresholds.

F.6 Calibration cost

Calibration requires running every calibration task through both cheap rungs and the reference. Table 11 sets that cost against the per-query saving: recording a calibration set costs 1.5M to 79.2M tokens and is repaid after 335 to 2,035 deployed queries at $\varepsilon = 10\%$ and 464 to 13,297 at 5%, the last where CLIMB saves little at that tolerance.

Routine arithmetic (gpt-oss-120b) **STOP AT L_0**
Task. Compute $\binom{31}{28}$. Gold: 4495.
Cheap rungs. L_0: 4495, $S_0 > -0.001$. **Decision → result.** Return 4495 at once. Correct.

Agreement settles a doubtful answer (gpt-oss-120b) **STOP AT L_1**
Task. Simplify $\tan 100^\circ + 4\sin 100^\circ$. Gold: $-\sqrt{3}$.
Cheap rungs. L_0: $-\sqrt{3}$, $S_0 = -0.256$, L_1: $-\sqrt{3}$. **Decision → result.** Too unsure to stop at L_0, but the rungs agree. Return $-\sqrt{3}$. Correct.

Team rescues a double miss (gpt-oss-120b) **ESCALATE TO TEAM**
Task. $24,000 earns 1% every two months for five years. Total to the nearest dollar. Gold: 32,348.
Cheap rungs. L_0: 32,340, $S_0 = -0.195$, L_1: 32,337. **Decision → result.** Too unsure to stop at L_0 and the rungs disagree. The team returns 32,348. Rescued.

Stopping protects a correct answer (gpt-oss-120b) **STOP AT L_1**
Task. Maximise $|(a+b)(a+c)(a+d)(b+c)(b+d)(c+d)|$ for distinct unit a, b, c, d summing to 0. Gold: 0.
Cheap rungs. L_0: 0, $S_0 = -0.366$, L_1: 0. **Decision → result.** The rungs agree. Return 0. The team would answer 8.

Escalation that does not help (gpt-oss-120b) **ESCALATE TO TEAM**
Task. Radius of a semicircle tangent inside a quarter circle of radius 14. Gold: $14/3$.
Cheap rungs. L_0: $14(\sqrt{2}-1)$, $S_0 = -0.185$, L_1: no answer. **Decision → result.** The team returns $14/(1+\sqrt{2})$, wrong at the full cost.

Figure 8: Five decision paths from the final MATH records. S_0 is the mean token log-probability of the direct answer. Escalation repairs some uncertain answers and fails on others, while stopping when the cheap rungs agree can keep a correct answer the team would miss.

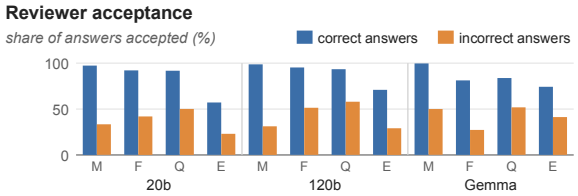


Figure 9: Share of the executor's correct and incorrect answers that the team's reviewer accepts, on twelve settings (M, F, Q, E: MATH, FRAMES, MuSiQue, BBEH).

Setting	n_{cal}	T_{cal}	$\Delta_{5\%}$	Payback	$\Delta_{10\%}$	Payback
20b MATH	200	1.7M	2,326	729	4,078	416
20b FRAMES	330	46.6M	31,579	1,474	62,914	740
20b MuSiQue	967	20.5M	8,460	2,424	10,402	1,972
20b BBEH	437	31.4M	22,362	1,402	46,714	671
120b MATH	200	1.5M	3,218	464	4,462	335
120b FRAMES	330	44.3M	31,584	1,404	57,694	768
120b MuSiQue	967	17.6M	6,682	2,629	8,858	1,983
120b BBEH	437	28.4M	13,251	2,140	32,000	886
Gemma MATH	200	4.1M	7,113	571	8,231	493
Gemma FRAMES	330	79.2M	24,272	3,264	66,523	1,191
Gemma MuSiQue	967	68.5M	15,630	4,381	33,643	2,035
Gemma BBEH	437	68.0M	5,114	13,297	45,338	1,500

Table 11: Calibration cost and payback. T_{cal}: tokens to record a calibration set (n_{cal} tasks, each run through both cheap rungs and the reference). Δ: mean tokens saved per deployed query at the given tolerance. Payback: T_{cal}/Δ queries (judge calls excluded).

G Case Studies

Figure 8 follows five MATH tasks through CLIMB: a confident early exit, a stop on the agreement of the two cheap rungs, a rescue by the team, a stop that keeps a correct answer the team would miss, and an escalation the team also gets wrong.

H Prompts

Executor system (L_0, L_1, pipeline). `You are an Executor. Answer concisely. End your reply with a single line: "ANSWER: <final answer>". For ANY computation, put Python code that prints the result in a ```python block; you will receive the output, then give the final answer.`
L_0 and L_1 user. `Question: {q} / Final answer.` (L_1 differs from L_0 only in its reasoning setting and sampling, and it does not see L_0's answer.)
Tool follow-up. `[Computation result] / {out} / Now give the final answer.`
Closing call (lowest reasoning setting). `Do not write any more code. Give the final answer now, ending with the line ANSWER: <answer>.`
Planner. System: `You are a Planner. Break down the question into sub-tasks.` User: `Question: {q} / Provide a step-by-step plan.`
Pipeline executor user. `Question: {q} / Plan: {plan} / Final answer.`
Reviewer. System: `You are a Reviewer. Validate the answer.` User: `Question: {q} / Proposed: {a} / Is the proposed answer correct? End your reply with a single line: VERDICT: YES or VERDICT: NO.`
Closing call for an empty review (reasoning at its lowest setting): `Question: {q} / Proposed: {a} / Is the proposed answer correct? Reply with a single line: VERDICT: YES or VERDICT: NO.`
Revision turn. `Reviewer feedback: {r} / Revise and give the final answer.`
Judge (FRAMES, MuSiQue). System: `You are an impartial grader of short factual answers.` User: `Question: {q} / Ground-truth answer: {gold} / Predicted answer: {pred} / Grade the predicted answer. It is CORRECT only if it gives the same answer as the ground truth: the same entity, or the same number, date or range (wording, units written differently, and extra explanation are fine). It is INCORRECT if any specific value differs (a different number, a narrower or wider range, a year instead of a full date, a different currency or quantity), if it answers only part of the question, hedges between several answers, or gives no answer. Reply with exactly one word: CORRECT or INCORRECT.`